

SUPER MARIO BROS PCG & DDA FRAMEWORK

COMPREHENSIVE TECHNICAL ARCHITECTURE, DATA FLOW DIAGRAMS (DFDs) & SYSTEM MODULES SPECIFICATION

1. Executive Overview & System Architecture

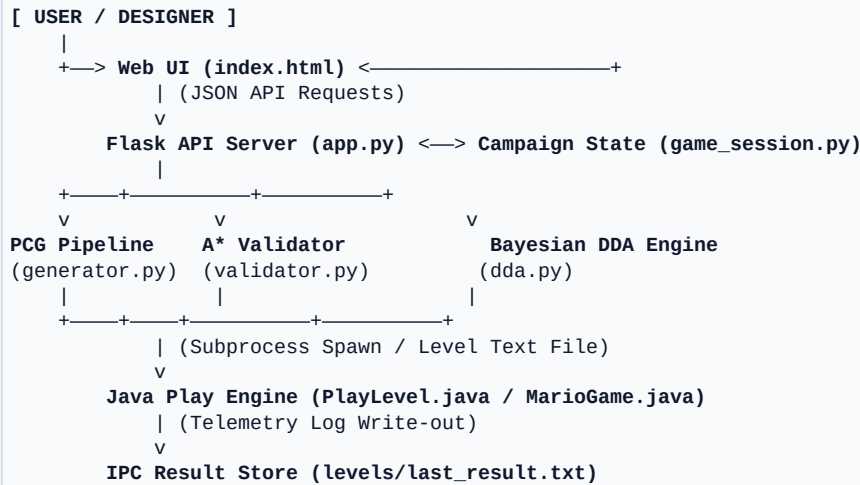
This document provides the definitive, complete technical specification of the **Super Mario Bros Trajectory-Conditioned Procedural Content Generation (PCG) & Bayesian Cumulative Dynamic Difficulty Adjustment (DDA) Framework**. The system transitions procedural level generation from static, single-level algorithms into an adaptive, multi-level campaign generator. Levels are generated dynamically to match designer-specified pacing curves (e.g. Linear Ramp, Spike & Recovery, Sawtooth Wave, Boss Rush) while continuously calibrating difficulty in real-time based on cumulative human gameplay telemetry.

1.1 Layered System Architecture

The framework is built upon a 4-tier decoupled architecture separating presentation, orchestration, procedural intelligence, and native game engine execution:

Architecture Layer	Components & Technologies	Key Responsibilities
1. Presentation Layer	HTML5, Vanilla CSS3 (Glassmorphism), Chart.js, HTML Canvas API	Interactive Difficulty Curve Designer, Tile Grid Visualizer, Proportional Spinner Loading Screen, Cumulative Death Zone Heatmap Canvas, Player Arcade Campaign Workspace.
2. REST API & Controller Layer	Python 3.13, Flask WSGI Engine, Subprocess IPC	RESTful endpoints for campaign generation, interactive Java game launching, death zone telemetry polling, and session progression state management.
3. PCG Intelligence Core	2nd-Order Markov Model, VGLC Trainer, Robin Baumgarten A* Solver, Bayesian DDA Engine	Trajectory-conditioned column sampling, playability validation ($\leq 5\%$ tolerance), stack block sanitization, exponential decay DDA weighting ($w_i = 0.70^{(n-1-i)}$).
4. Native Execution Layer	Java 17, Swing AWT Window Framework, Mario-AI-Framework Engine	3.5x scaled interactive human play window, live A* agent visualizer, real-time physics engine, in-game Swing HUD (Lives/Coins), IPC result logger (last_result.txt).

1.2 Structural Component Interaction Diagram



2. Data Flow Diagrams (DFDs)

Data Flow Diagrams model the movement, transformation, and storage of data across the system. The following diagrams document the system from high-level context (Level 0) down to granular subsystem process flows (Level 2).

2.1 DFD Level 0: Context Diagram

The Context Diagram defines the global system boundary, external entities (Game Designer, Human Player), and primary input/output data streams.

External Entity	Input Data Stream to System	Output Data Stream from System
Game Designer (Web UI User)	- Target Difficulty Vector $D = [d_1 \dots d_N]$ (5–95) - Curve Presets (Linear, Spike, Wave, Boss) - Simulated Skill Profile (Novice, Average, Pro) - Level Count N (3 to 10)	- Rendered Chart.js Wave Graph - Solvability & Difficulty Metrics - Tile Grid Visualizer & Inspection - Real-Time Loading Progress Spinner - Session DDA Telemetry Summary
Human Player (Arcade Mode User)	- Selected Unlocked Level Index - Real-Time Keyboard Controls (Arrows, S, A) - Campaign Reset Command (3 Lives Lost)	3.5x Scaled Java Game Window - Live Top-Left HUD (Lives: 3/2/1, Coins) - Flagpole Course Clear Unlock Signal - Cumulative Death Zone Heatmap Overlay

2.2 DFD Level 1: System-Level Data Flow Diagram

Level 1 decomposes Process 0.0 into 5 core functional processes, highlighting data transformations and data store interactions:

Process ID & Name	Input Data Flows	Data Transformation / Operation	Output Data Flows & Data Stores
1.0 Sequence Generator	Target Vector D , Markov Matrix Store	Conditions 2nd-order column transitions; inserts 8-step stair intros/outros; executes <code>_fix_stacked_special_blocks</code> .	Candidate Level Text → <i>Data Store: Candidate Buffer</i>
2.0 Solvability Validator	Candidate Level Text	Spawns Robin Baumgarten A* solver; measures completion time, jump heights, gap clearance; checks $\leq 5\%$ tolerance.	Validated Level Text, Est. Diff → <i>Data Store: Active Level Store</i>
3.0 API & Session Controller	UI Requests, Player Selections	Orchestrates pipeline execution; tracks active level index, unlocked maps, 3-life system, and coin counts.	JSON API Responses → <i>Data Store: Session State Store</i>
4.0 Java Play Engine	Level File, Mario State, Lives	Executes real-time physics loop; renders 3.5x AWT window & HUD; detects death/win events.	SESSION_UPDATE: string → <i>Data Store: last_result.txt</i>
5.0 Bayesian DDA Engine	Telemetry Data, Session History	Calculates weighted performance P_{cum} using $w_i = 0.70^{(n-1-i)}$; computes bounded shift Δd in $[-0.10, +0.10]$.	Adjusted Target $d'_{\{k+1\}}$ → <i>Data Store: DDA History Store</i>

2.3 DFD Level 2: Detailed Process Flow Breakdown

Level 2 details the internal sub-process mechanics of critical system operations:

- **DFD 2.1 Generation & Sanitization Sub-Process:**
- 1.1 Receive d_k → 1.2 Query VGLC 2nd-Order Markov Matrix $P(C_i | C_{\{i-1\}}, C_{\{i-2\}}, d_k)$ → 1.3 Generate raw 220-column grid → 1.4 Inject 8-step upward intro staircase & 8-step flagpole outro staircase with castle walk → 1.5 Scan columns for special blocks ($?$, Q , 1 , 2) stacked on obstacles & execute `_fix_stacked_special_blocks()` to shift up/clear → 1.6 Populate Goomba, Koopa, Spiky, Bullet Cannon, and Piranha Pipe decorations.
- **DFD 2.2 Solvability Validation Sub-Process:**

2.1 Convert level text grid to Java MarioLevel → 2.2 Launch background A* search agent (200s limit) → 2.3 Evaluate completion ($\geq 85\%$ or WIN status) & calculate estimated difficulty d_{est} → 2.4 Check constraint $|d_{est} - d_k| \leq 0.05$ → If Pass: return validated level; If Fail: increment attempt counter (max 50) & re-seed generator.

• **DFD 2.3 Real-Time DDA & Telemetry Sub-Process:**

3.1 Player completes attempt in Java window → 3.2 PlayLevel.java writes SESSION_UPDATE:WON=b:MODE=m:LIVES=l:COINS=c:COMPLETION=pct:KILLS=k:HURTS=h:JUMPS=j to levels/last_result.txt → 3.3 app.py parses key-value pairs & extracts death coordinates (X, Y) → 3.4 Append death marker to ACTIVE_DEATH_ZONES[current_lv] → 3.5 dda.py calculates P_cum & shifts target d_k (on death) or d_{k+1} (on win) → 3.6 Web UI re-renders telemetry cards & Canvas heatmap overlay.

3. Detailed System Modules Specification

This section provides an exhaustive line-item specification of all software modules composing the system.

3.1 Module 1: Web Engine & REST API Controller (app.py)

app.py serves as the central API gateway and web application server built using Python Flask. It manages web routes, asynchronous Java subprocess invocations, IPC telemetry file parsing, and death zone registry state.

Primary API Endpoints:

Endpoint Route		HTTP Method	Function & Data Payload
	/	GET	Renders main HTML5 Web UI visualizer dashboard (templates/index.html).
	/api/generate-sequence	POST	Accepts {curve: [d1..dN], player_skill: 'average', max_variance: 0.10}. Executes full campaign pipeline, returning level grids, solvability metrics, and telemetry.
	/api/generate-sequence-stream	POST	Server-Sent Events (SSE) streaming real-time compilation and validation progress (progress, level_complete, complete events).
	/api/play-human-level	POST	Accepts {level_index: k, mario_mode: m, lives: l, coins: c, level_text: t}. Enforces lives > 0 and generated map guards; spawns interactive 3.5x Java Swing window executing PlayLevel.java.
	/api/watch-ai-play	POST	Spawns visual 3.5x Java Swing window displaying Robin Baumgarten A* agent actively solving the validated level.
	/api/check-session-result	POST	Polls levels/last_result.txt. Parses key-value pairs (WON=true/false, LIVES=l, COMPLETION=pct), records cumulative death coordinates in ACTIVE_DEATH_ZONES, applies DDA spatial tweaks on death (retaining level slot k) or generates pre-adjusted level k+1 on win.
	/api/telemetry/death-zone/<idx>	GET	Returns cumulative death points (X, Y), struggle reasons, and death counts for level index idx across all playthroughs.

3.2 Module 2: Procedural Generator & Markov Model (generator.py & vglc_model.py)

This module constructs candidate level grids conditioned on target difficulty metrics. It uses a 2nd-order Markov chain model trained on 31 VGLC Super Mario Bros levels (vglc_trainer.py).

Key Procedural Generation Rules & Algorithms:

- **Conditioned Column Sampling:** Computes transition probability matrix $P(C_i | C_{i-1}, C_{i-2}, d_k)$ where C_i represents a 16-tile vertical column.
- **Authentic Level Intros:** Columns 0–12 feature flat ground ('X') ensuring safe player spawning.
- **Authentic Level Outros:** Columns (W-30) to (W-15) feature an 8-step upward staircase leading to a flagpole placed 4–6 tiles away, followed by an automated walk to a castle door.

- **Unhittable Stack Block Sanitizer (`_fix_stacked_special_blocks`):** Scans columns for reward blocks ('?', 'Q', '1', '2') stacked directly on top of solid blocks ('S', '#', '%') and shifts them up to guarantee jumping clearance height.
- **Targeted DDA Spatial Micro-Adjuster (`tweak_level_for_dda`):** On player death retries, modifies the exact level layout by injecting safety stepping platforms across pits, demoting aggressive enemies near the failure column `X_fail`, and inserting power-up ? blocks 5 columns prior to failure point.
- **Enemy & Pipe Decoration Post-Processing:** Dynamically places Goombas ('g'), Koopas ('k/'r'), Flying Koopas ('K/'R'), Spikies ('y/'Y'), Bullet Bill Cannons ('B/'b'), and Piranha Flower Pipes ('T').

3.3 Module 3: A* Playability Validator (`validator.py`)

`validator.py` acts as an automated quality assurance engine. It invokes the Robin Baumgarten A* search agent (from the Mario AI Framework) in headless background mode to test if Mario can successfully navigate from start to finish within 200 seconds.

- **Solvability Verification:** Measures completion percentage ($\geq 85\%$ required) and game status (WIN).
- **Difficulty Calibration:** Calculates structural jump difficulty d_{est} based on required jump heights, gap widths, and enemy densities.
- **Automated Retries:** If $|d_{est} - d_k| > 0.05$ or the level is unsolvable, the generator automatically re-seeds and retries up to 50 times.

3.4 Module 4: Bayesian Cumulative DDA Engine (`dda.py`)

The DDA engine dynamically adapts level difficulty based on cumulative human performance telemetry across all preceding levels played in the active session.

Mathematical Formulations:

1. Individual Level Performance Index (P_i):

$$P_i = 0.40 * Won_i + 0.30 * CompletionPct_i + 0.15 * (1 - \min(1, Hurts_i / 5)) + 0.15 * (Mode_i / 2)$$

2. Exponential Decay Session Weighting (w_i):

$$w_i = 0.70^{(n - 1 - i)} \text{ for level } i \text{ in } \{0 \dots n-1\}$$

3. Cumulative Performance Index (P_{cum}):

$$P_{cum} = (\sum_{i=0}^{n-1} w_i * P_i) / (\sum_{i=0}^{n-1} w_i)$$

4. Bounded Difficulty Target Shift (Δd):

$$\Delta d = \text{Clamp}(\alpha * (P_{cum} - P_{target}), -0.10, +0.10) \text{ where } \alpha = 0.25$$

- **Scoped Level Tuning & Slot Retention:** When a player dies on Level k , the session strictly locks to Level slot k with a micro-adjusted layout. When Level k is cleared, DDA shifts target Level $k+1$ before play begins.

3.5 Module 5: Campaign & Session State Manager (`game_session.py`)

`game_session.py` manages player arcade progression state, life system (3 lives bounded at 0), coin accumulation (100 coins = +1 life), and campaign level preservation.

- **Campaign Sequence Generation:** Generates initial level sequences matching designer curve targets with 0.0% initial DDA shift without simulated skill guessing.
- **Arcade Level Preservation:** When a player loses 3 lives and resets the campaign, previously generated level layouts are preserved so players replay the exact same levels unlocked earlier.
- **Life System Lower Bounding & Game Over:** Bounded at 0 lives; on 3rd death, execution halts with Game Over status until manual reset.

3.6 Module 6: Web Interface & Visualizer (`templates/index.html`)

The frontend is built using Vanilla CSS glassmorphism aesthetics, Chart.js graphs, HTML Canvas overlays, and modal progress animations.

- **Chart.js Wave Curve Designer:** Shows ONLY designer-intended curve on initial load; gates actual difficulty and DDA-adjusted curves behind generation completion.
- **Level Map Visualizer Inactive Lifecycle:** Displays clean placeholder guidance and disabled buttons prior to generation; activates upon validated campaign return.
- **Bounded Horizontal Scrolling (.level-view-container):** Locks viewing box to grid height (260px) with custom glowing horizontal scrollbars across all 220 columns.
- **Synchronized Canvas Coordinate Space (.level-canvas-wrapper):** Locks `#heatmapCanvas` and `#tileGrid` in exact 1:1 pixel coordinate alignment, accurately rendering death skull markers (■) and AI playback across the full level width.
- **Arcade Progression Workspace & HUD:** Level selection buttons (``World 1-1 ✓``, ``World 1-2 ■``), live life counter (``♥■3``), and Game Over play locking.

3.7 Module 7: Java Swing Engine Framework (Mario-AI-Framework)

The native execution engine is implemented in Java 17 and Swing AWT.

- **PlayLevel.java:** Main entry point handling command-line arguments (level path, mode, state, lives, coins), clamping `newLives = Math.max(0, newLives)` on death.
- **In-Game Swing HUD:** Passes player lives and coin counts into `MarioWorld`, rendering remaining lives on the top-left HUD.
- **IPC Telemetry Logger:** Writes ``SESSION_UPDATE:WON=b:MODE=m:LIVES=l:COINS=c:COMPLETION=pct:KILLS=k:HURTS=h:JUMPS=j`` to ``levels/last_result.txt`` upon game over or course clear.

4. System Verification & Audit System

All structural modifications, API updates, and algorithm enhancements are systematically validated and recorded in date-stamped logs inside the `Remarks/` directory (e.g. `Remarks/2026-08-06.txt`).